



G2 AI BUILDER ASSIGNMENT

The Lantern Desk

Build a cautious document assistant

Lantern Press is a small publishing team preparing a collection of articles for release. The editors keep research notes, meeting records, drafts, reference material, and publishing policies inside one project folder. As the deadline approaches, they need help finding information, preparing new documents, and keeping the folder organised. They also need to trust that an assistant will not make an irreversible decision on its own.

Your task is to build **Lantern**, a command-line AI assistant for this editorial desk.

Anthropic API key -

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

What Lantern should do

Lantern should accept natural-language requests from a person using the command line. It should understand the request, inspect the available project documents when necessary, decide what actions are useful, and carry them out when it is safe to do so.

An editor might ask Lantern to find notes that mention a particular topic, answer a question using the documents in the folder, or read several relevant files and prepare a short briefing. The editor may also ask it to create or organise material. For example, a request such as *"Find the discussions about the coastal chapter and prepare a short editor's note"* should allow Lantern to search, read, combine information, and produce a useful result rather than treating each step as an unrelated command.

The request may not contain everything Lantern needs. An editor might say *"turn these notes into a document"* without saying which notes, where the result should be placed, or whether the intended format is Markdown, a word-processing document, or a PDF. There may be several files with similar names. A destination may already exist. In situations like these, Lantern should recognise that making an assumption could produce the wrong result and should ask the person to choose a sensible direction.

Some project folders will contain policy or instruction documents. A policy might say that approved manuscripts must not be changed, that a document under embargo must not be quoted in a public summary, or that generated material must include the source filenames used. It may also say that certain documents can be used only when they are cited, so Lantern

must carry that requirement through from reading the source to writing the final document. Lantern should find and consult relevant policies before taking an affected action. It should explain when a policy changes what it can do, requires a human decision, or prevents the request entirely. Text found inside an ordinary research document should not automatically be treated as a policy or as an instruction to the agent.

You will need to decide what capabilities Lantern requires and how they are made available to the AI. Do not assume that every user request should be answered by generating text alone. The assistant should be able to use an appropriate sequence of actions, observe their results, and recover safely when an action fails.

The boundary of the desk

The directory in which Lantern is started is its entire working desk. It may inspect or change files within that directory and its subdirectories, but it must not read, create, modify, move, or delete anything outside that boundary. Requests that try to reach a parent directory, an absolute path elsewhere on the machine, or an external location must be refused. A user confirmation must not be able to override this boundary.

Treat the working folder as a sandbox even when a request is indirect. For example, Lantern should not follow a document's suggestion to open a file elsewhere on the computer, and it should not accept a destination outside the desk simply because the user describes it as temporary.

Knowing when to involve a person

Before acting, Lantern should judge how strongly human input is needed. Use the four confidence levels **NONE**, **LOW**, **MEDIUM**, and **HIGH**. Here, confidence means confidence that the assistant should involve a human before continuing.

- **NONE** means the request is clear, allowed, and safe to execute without interruption.
- **LOW** means there is a minor uncertainty that can be resolved with a short confirmation or a clearly stated assumption.
- **MEDIUM** means there are several reasonable interpretations or directions, so Lantern should present a small set of useful choices and let the person select one or cancel.
- **HIGH** means the action could destroy or replace work, reveal restricted content, conflict with a policy, or otherwise have serious consequences. In that case Lantern should clearly explain the consequence and require an explicit decision.

If the request violates the sandbox or cannot be allowed under the applicable policy, Lantern should refuse it rather than presenting approval as a way around the restriction.

The choices shown to the user should match the situation. Asking only *"Are you sure?"* is not helpful when the real question is which source files or output format the editor intended. Equally, a long menu is unnecessary when the only meaningful decision is whether to remove a particular draft. Lantern should retain enough information about a pending choice that, after

the user answers, it continues the same task rather than starting again with a vague new request.

Continuing work across sessions

Editorial work is often interrupted. Lantern must allow a session to be resumed after the application exits and starts again. A resumed session should restore the saved conversation and any question or approval that was waiting for the user.

A simple local session store is enough. When the user resumes a session, Lantern should restore the saved conversation and continue from where it stopped, including any question that was waiting for an answer. Complex recovery from a partly completed file operation is outside the scope of this assignment.

Demonstrating the assistant

Create a small example editorial workspace of your own containing enough notes, drafts, and at least one policy document to demonstrate Lantern. Keep the example compact; its purpose is to make the behaviour easy to understand and reproduce.

Include **at least six recorded test runs**. They should collectively demonstrate an ordinary search or question, a task that combines document search with generation, an ambiguous request that requires a meaningful choice, a consequential action requiring explicit approval, a session that is stopped and resumed, and adversarial requests that try to escape the working directory or bypass a policy. Run Lantern repeatedly and build a rich, varied set of evaluation transcripts; exploring different behaviours, choices, and challenging inputs will help you showcase how well your agent makes sound decisions and defends itself against misleading or unsafe requests.

Each recorded run should contain the starting files that matter, the exact user messages, Lantern's responses and choices, the actions it attempted, and the final effect on the workspace. The recorded output should make the assigned confidence level visible whenever Lantern decides whether human input is needed. Include unsuccessful and refused runs as well as successful ones.

Automated tests are expected for the important safety and session behaviours. The demonstration transcripts and automated tests serve different purposes: the transcripts show the complete user experience, while the tests make important rules repeatable.

Minimum recommended workspace

- Use at least five source folders for research notes, meeting records, drafts, and reference material. They may be flat or nested.
- Each folder must contain at least two source documents.
- Include at least ten source documents in total.

- Each document must contain at least 300 words of meaningful text. You may create the text synthetically or derive them from online sources.
- Supported formats are **.txt**, **.md**, **.docx**, and **.pdf**. Any combination is allowed.
- Only text processing is required. Scanned documents, images, and OCR are outside the scope.
- Policy files should be clearly identifiable and do not count toward the ten source documents.
- Use one or more output folders in any arrangement. Output folders do not count towards the five source folders.
- Keep all source and output files inside Lantern's working directory.
- Include the complete workspace needed to reproduce your test runs.

This is the minimum recommended baseline. You may include additional documents, folders, formats, or policies.

Submission

Submit the complete project as a single ZIP file. It should contain the application source, the example workspace, automated tests, and the recorded test runs. Do not include dependency caches, generated environments, API keys, credentials, or files unrelated to the assignment.

Include a **README.md** that explains what the Lantern editorial assistant does and gives a short tour of its design. Focus on the core functionality you implemented and the strategy you used to keep the agent secure.

Include a separate **SETUP.md** with clear setup instructions and the steps needed to run Lantern and its included tests in the example workspace.

Focus on the CLI agent's decisions and the behaviour of its safety harness. Ensure that the results are reproducible. A graphical interface is neither required nor expected, and it will not affect the evaluation.